JavaScript

What is JavaScript?

JavaScript is a programming language that runs in the browser. It makes web pages interactive and dynamic responding to user actions (clicks, typing, scrolling), updating page content without reloading, validating forms, and communicating with servers.

Unlike HTML and CSS which are static, JavaScript allows a page to react to what the user does in real time. It is the only programming language that runs natively inside a web browser.

Variables & Data Types

Variables store values that can be used throughout a script. JavaScript provides three ways to declare them:

- **let** : block-scoped; can be reassigned (preferred for most variables).
- **const** : block-scoped constant; cannot be reassigned after declaration.
- **var** : function-scoped; older style, generally avoided in modern JS.

```
// String
let name = "Raunak Poudel";

// Number (constant)
const age = 20;

// Boolean
let isStudent = true;

// Template literal — embed variables inside a string
let greeting = `Hello, ${name}! You are ${age} years old.`;

// Array
let languages = ["HTML", "CSS", "JavaScript"];
console.log(languages[0]); // Output: "HTML"

// Object
let student = {
    name: "Raunak",
    email: "raunakpoudel0@gmail.com",
    year: 1
};
console.log(student.name); // Output: "Raunak"
```

Functions

Functions are reusable blocks of code that perform a specific task. They are defined once and called multiple times with different inputs.

```
// Function declaration
function greet(name) {
    return "Hello, " + name + "!";
}

// Arrow function (shorter modern syntax)
```

```
const add = (a, b) => a + b;

// Calling functions
console.log(greet("Raunak")); // "Hello, Raunak!"
console.log(add(5, 3));       // 8
```

DOM Manipulation

The Document Object Model (DOM) is the browser's representation of an HTML page as a tree of objects. JavaScript can select, read, and modify these objects to dynamically update the page.

```
// Select elements
const heading = document.getElementById("main-title");
const allButtons = document.querySelectorAll(".btn");

// Change content and style
heading.innerHTML = "Updated Title";
heading.style.color = "blue";

// Listen for a button click
document.getElementById("myBtn").addEventListener("click", function() {
    alert("Button was clicked!");
});
```

Demo 1 Counter (DOM + Events)

This demonstrates DOM manipulation and event handling. Three buttons update a counter displayed on screen. The count variable lives in JavaScript memory and the DOM element is updated on every click.

```
let count = 0;

function increment() {
    count++;
    document.getElementById("count-display").textContent = count;
    document.getElementById("count-msg").textContent = "Incremented to " + count;
}

function decrement() {
    count--;
    document.getElementById("count-display").textContent = count;
    document.getElementById("count-msg").textContent = "Decremented to " + count;
}

function resetCount() {
    count = 0;
    document.getElementById("count-display").textContent = count;
    document.getElementById("count-msg").textContent = "Counter reset.";
}
```

The HTML buttons call these functions via onclick:

```
<h2 id="count-display">0</h2>
<p id="count-msg"></p>
<button onclick="decrement()">- Decrement</button>
<button onclick="resetCount()">Reset</button>
```

```
<button onclick="increment()">+ Increment</button>
```

Demo 2 To-Do List (Arrays + DOM)

Tasks are stored in a JavaScript array. Adding an item pushes it to the array; removing one filters it out. The renderTodos() function rebuilds the list HTML every time the array changes.

```
let todos = [];

function addTodo() {
  const input = document.getElementById("todo-input");
  const text = input.value.trim();
  if (!text) return;
  todos.push({ id: Date.now(), text });
  input.value = "";
  renderTodos();
}

function removeTodo(id) {
  todos = todos.filter(t => t.id !== id);
  renderTodos();
}

function renderTodos() {
  const list = document.getElementById("todo-list");
  list.innerHTML = todos.map(t =>
    `<li class="list-group-item d-flex justify-content-between">
      ${t.text}
      <button onclick="removeTodo(${t.id})"
        class="btn btn-sm btn-outline-danger">Remove</button>
    </li>`
  ).join("");
}
```

Demo 3 — Fetch API (Async/Await)

The Fetch API allows JavaScript to make HTTP requests to external servers without reloading the page. The async/await syntax makes asynchronous code readable and easy to follow.

```
async function fetchJoke() {
  const output = document.getElementById("joke-output");
  output.textContent = "Loading...";

  try {
    const response = await fetch(
      "https://official-joke-api.appspot.com/random_joke"
    );
    const data = await response.json();
    output.innerHTML =
      `<strong>${data.setup}</strong><br>${data.punchline}`;

  } catch (error) {
    output.textContent = "Error loading joke. Check your connection.";
  }
}
```

How each part works:

- **fetch()** : sends a GET request to the public joke API URL.
- **await** : pauses execution until the server responds, without blocking the browser.
- **response.json()** : parses the raw HTTP response as a JavaScript object.
- **try/catch** : catches network errors so the page doesn't crash.

Loops & Conditionals

Loops repeat a block of code, and conditionals execute code only when a condition is true.

```
// For loop
for (let i = 1; i <= 5; i++) {
    console.log("Number: " + i);
}

// While loop
let x = 0;
while (x < 3) {
    console.log(x);
    x++;
}

// If / else if / else
const score = 72;
if (score >= 70) {
    console.log("Pass");
} else if (score >= 50) {
    console.log("Borderline");
} else {
    console.log("Fail");
}

// Ternary operator (shorthand if/else on one line)
const result = score >= 70 ? "Pass" : "Fail";
```